

DG-202612 文旅搬运赛题变更说明

一、说明目的

因赛题难度、任务连续性和仿真场景设置调整，“面向物品识别与搬运的文旅机器人关键技术研究”赛题的任务内容、场景随机规则和评测接口较原比赛方案有所变化。

本说明面向参赛选手发布，用于说明当前正式提供的 Docker 镜像与原比赛方案之间的主要差异。参赛选手应以本说明、随镜像发布的《文旅搬运 README》和指定版本 Docker 镜像为准开展开发。

本次提供的镜像为：

- Server: `material_sorting:offline-server`
- Client: `material_sorting:offline-client`

除本说明明确调整的内容外，参赛资格、作品材料、赛程安排、提交方式、奖项和联系方式等继续按照原比赛方案及后续官方通知执行。

二、任务内容调整

2.1 由两项任务调整为三步连续重排任务

原比赛方案中的两项搬运任务，调整为一局内依次执行的三步关联任务：

1. 抓取桌面左侧或右侧的指定彩色箱子，放到货架空层；
2. 抓取货架中的指定彩色箱子，放到第一个箱子原来所在的桌面侧边位置；
3. 抓取白色正方体顶部的指定彩色箱子，放到货架白色长方体障碍物的左边。

后一项任务使用同一局开局时生成的随机布局信息。参赛程序应连续完成三项任务，不应假定每项任务会通过重启程序或恢复固定布局单独进行。

2.2 任务指令改为 ROS 2 结构化指令

Server 通过以下话题提供任务信息：

Topic	Type	说明
<code>/material/instruction</code>	<code>std_msgs/msg/String</code>	本局三项任务的 JSON 指令列表

Topic	Type	说明
/referee/taskinfo	std_msgs/msg/String	当前任务中文说明
/referee/gameinfo	std_msgs/msg/String	当前时间、分数、任务和尝试进度
/referee/score	std_msgs/msg/Int32	当前总分

/material/instruction 中包含 task、instruction、target_color、target_body、place_world、place_type、place_radius 等任务字段。参赛程序可以读取并使用这些正式发布字段，同时结合视觉检测结果判断目标颜色和当前物品位置。

参赛程序不得把粉色、黄色、褐色箱子的任务顺序、初始位置、桌面左右位置或货架层数写死。

三、场景和随机规则调整

3.1 可抓取物品

场景中有粉色、黄色和褐色三个彩色长方体箱子，每个尺寸为 $24 \times 16 \times 19$ cm。三个彩色箱子是当前任务中可抓取和搬运的对象。

原比赛方案中的中文“棕色包装盒”，在当前指令中显示为“褐色方块”；视觉类别标识仍为：

- pink
- yellow
- brown

3.2 固定参照物

当前场景保留：

- 桌面白色正方体障碍物；
- 货架白色长方体障碍物。

原比赛方案中的圆形工具桶不再作为当前场景道具。

3.3 正式随机布局

正式 Server 每次启动默认开启随机布局：

1. 三个彩色箱子随机分配到“桌面侧边、白色正方体顶部、货架”三个槽位；
2. 桌面侧边箱子随机位于白色正方体左侧或右侧；
3. 货架彩色箱子随机位于从下往上数第一、第二或第三层；
4. 白色长方体障碍物随机位于前三层中与彩色箱子不同的另一层；

5. 剩余一层为空层，作为任务一的放置目标。

正式评测参数为：

```
MATERIAL_RANDOMIZE=1
MATERIAL_ENABLE_SCORE=1
MATERIAL_ENABLE_RENDER=1
MATERIAL_USE_GS=1
```

正式运行通常不设置 `MATERIAL_SEED`，由 Server 自动生成随机种子。需要复现某次布局时，可由赛务人员使用该次种子重新启动。

四、比赛流程调整

1. 每局开始时，Server 生成随机布局并发布三项任务指令。
2. 机器人从出发区开始执行任务一。
3. 机器人离开结束区后，当前任务的一次尝试开始。
4. 完成当前任务并返回结束区后，系统结算本次尝试。
5. 当前任务正确放置成功后，系统进入下一项任务。
6. 每项任务最多有三次机会，按该任务单次最高得分计入总分。
7. 某项任务三次机会用尽后，系统进入下一项任务。
8. 三次尝试之间，场景中的物品不会自动恢复位置，参赛程序应根据实时感知处理物品的当前位置。
9. 三项任务全部结算或仿真时间达到 600 秒，本局结束。

比赛时限仍为 10 分钟，但当前裁判使用 MuJoCo 仿真时间计时。

五、评分调整

当前 Docker Server 内置自动裁判，三个任务的原始分值如下：

任务	触碰目标	夹持并搬离	正确放置	安全返回	满分
任务一	10	10	10	10	40
任务二	20	10	20	10	60
任务三	20	10	20	10	60
总分					160

评分要点：

1. 触碰、搬离、放置和返回按顺序判定；
2. “夹持并搬离”要求目标箱同时与左右夹持链路接触，并相对开局位置产生不少于 0.20 m 的水平位移；
3. 正确放置要求目标箱已经松开、运动稳定，并进入指令目标点的允许范围；
4. 机器人与货架或外围墙发生碰撞时，本次尝试不获得“安全返回”分；
5. 已获得的触碰、搬离和放置分不会仅因上述结构碰撞被清除；
6. 目标箱在搬离后掉落至规定高度以下，会结算当前尝试；
7. 每项任务最多三次机会，取该任务单次最高分。

运行时可通过 `/referee/score` 查看当前总分。Server 退出时会生成 `referee_results_<timestamp>.json` 结果文件。

六、仿真和通信环境调整

当前运行环境为：

- Ubuntu 22.04;
- ROS 2 Humble;
- Cyclone DDS;
- CUDA 12.8;
- PyTorch 2.8.0+cu128;
- torchvision 0.23.0+cu128;
- MuJoCo;
- 3D Gaussian Splatting。

正式启动时 Server 和 Client 均使用：

```
ROS_DOMAIN_ID=99
RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

Server 和参赛程序的 `ROS_DOMAIN_ID` 必须一致。请直接使用 README 中的正式 Server 和 Client 启动命令，不要依赖镜像内未覆盖的环境变量默认值。

当前正式配置提供头部 RGB-D、左右腕部 RGB、里程计、关节状态和 TF；不提供二维激光雷达话题。

七、Client 镜像和参赛程序说明

`material_sorting:offline-client` 是参赛程序的运行环境。使用时应将选手项目目录挂载到：

```
/workspace/baseline
```

然后在 Client 容器内启动参赛程序。示例：

```
cd /workspace/baseline  
python3 your_client.py
```

参赛程序的内部文件组织可由团队自行设计，但启动后的程序应能在同一局中读取随机指令并连续处理全部三项任务。作品正式提交材料仍按原比赛方案和后续提交通知执行。

参赛镜像或项目中应包含离线运行所需的代码、模型、权重和其他资源，不应在正式评测期间依赖互联网下载。

八、示例程序适用范围

Docker 资料中的示例程序用于展示：

- ROS 2 话题订阅和控制发布方式；
- RGB-D 与 YOLO 感知节点的组织方式；
- 机器人导航、关节控制和抓放状态机的基本写法；
- 参赛文件的目录和启动形式。

示例程序不代表完整参赛方案，也不保证直接完成正式随机赛题。

其中 `client_task_1.py` 是固定调试场景下的任务一示例，使用时必须设置：

```
MATERIAL_RANDOMIZE=0
```

正式随机场景使用 `MATERIAL_RANDOMIZE=1`。参赛团队需要根据 `/material/instruction` 自行完成随机颜色、随机位置、随机货架层位和三项连续任务的适配。

示例感知节点发布的 `/material/detections` 不是 Server 原生裁判话题。参赛团队可以使用随附 YOLO 示例，也可以自行实现物品检测与定位算法。

示例代码只用于学习和调试，不作为评分规则依据。评分以正式 Server 内置裁判结果为准。